

AI ASSISTED CODING ASSIGNMENT NO:13.3

NAME : SIRIVELLA SANJANA

ROLL NO : 2403A510D4

BATCH NO : 05

Task Description #1 – Remove Repetition

Task: Provide AI with the following redundant code and ask it to refactor

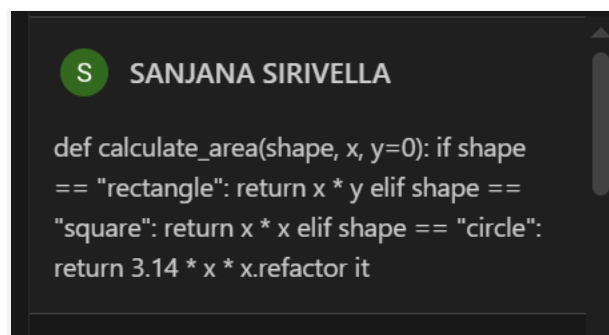
Python Code

```
def calculate_area(shape, x, y=0):  
    if shape == "rectangle":  
        return x * y  
    elif shape == "square":  
        return x * x  
    elif shape == "circle":  
        return 3.14 * x * x
```

Expected Output

- Refactored version with dictionary-based dispatch or separate functions.
- Cleaner and modular design

PROMPT:



CODE:

```
m.py portfolio.html 2q.html studentjs an2.1(2).py an2.1(3).py 13.3(1).py x
13.3(1).py > ...
1 import math
2 from typing import Callable, Dict, Union
3
4 # Define a type for numeric values for clarity
5 Numeric = Union[int, float]
6
7 def _calculate_rectangle_area(length: Numeric, width: Numeric) -> Numeric:
8     """Calculates the area of a rectangle."""
9     return length * width
10
11 def _calculate_square_area(side: Numeric, *args) -> Numeric:
12     """Calculates the area of a square."""
13     return side * side
14
15 def _calculate_circle_area(radius: Numeric, *args) -> Numeric:
16     """Calculates the area of a circle."""
17     return math.pi * (radius ** 2)
18
19 # A dictionary to map shape names to their respective calculation functions.
20 # This is a form of dispatch table.
21 _SHAPE_FUNCTIONS: Dict[str, Callable[..., Numeric]] = {
22     "rectangle": _calculate_rectangle_area,
23     "square": _calculate_square_area,
24     "circle": _calculate_circle_area,
25 }
26
27 def calculate_area(shape: str, *dims: Numeric) -> Numeric:
28     """
29     Calculates the area of a given geometric shape.
30
31     This function uses a dispatch table to call the appropriate
32     area calculation function based on the shape name.
33
34     Args:
35         shape (str): The name of the shape (e.g., 'rectangle', 'square', 'circle').
36         *dims (Numeric): The dimensions of the shape.
37             - For 'rectangle': (length, width)
38             - For 'square': (side,)
39             - For 'circle': (radius,)
40
41     Returns:
42         Numeric: The calculated area of the shape.
43
44     Raises:
45         ValueError: If the shape is not supported.
46         TypeError: If the wrong number of dimensions is provided for a shape.
47
48     Example:
49         >>> calculate_area("rectangle", 10, 5)
50         50
51         >>> calculate_area("square", 4)
52         16
53         >>> calculate_area("circle", 3)
54         28.274333882308138
55     """
```

```
13.3(1).py > ...
26
27 def calculate_area(shape: str, *dims: Numeric) -> Numeric:
28     """
29     Calculates the area of a given geometric shape.
30
31     This function uses a dispatch table to call the appropriate
32     area calculation function based on the shape name.
33
34     Args:
35         shape (str): The name of the shape (e.g., 'rectangle', 'square', 'circle').
36         *dims (Numeric): The dimensions of the shape.
37             - For 'rectangle': (length, width)
38             - For 'square': (side,)
39             - For 'circle': (radius,)
40
41     Returns:
42         Numeric: The calculated area of the shape.
43
44     Raises:
45         ValueError: If the shape is not supported.
46         TypeError: If the wrong number of dimensions is provided for a shape.
47
48     Example:
49         >>> calculate_area("rectangle", 10, 5)
50         50
51         >>> calculate_area("square", 4)
52         16
53         >>> calculate_area("circle", 3)
54         28.274333882308138
55     """
```

```

13.3(1).py > ...
27 def calculate_area(shape: str, *dims: Numeric) -> Numeric:
56     shape_lower = shape.lower()
57
58     if shape_lower not in _SHAPE_FUNCTIONS:
59         raise ValueError(
60             f"Unsupported shape: '{shape}'. "
61             f"Supported shapes are: {list(_SHAPE_FUNCTIONS.keys())}"
62         )
63
64     calculation_func = _SHAPE_FUNCTIONS[shape_lower]
65
66     try:
67         return calculation_func(*dims)
68     except TypeError:
69         # This helps provide a better error message if the user passes
70         # the wrong number of arguments (e.g., calculate_area("square", 4, 5))
71         raise TypeError(f"Incorrect number of dimensions provided for shape '{shape}'.")
72
73 # --- Example Usage ---
74 if __name__ == "__main__":
75     # Correct usage
76     rect_area = calculate_area("rectangle", 10, 4)
77     print(f"Area of rectangle (10x4): {rect_area}")
78
79     sq_area = calculate_area("square", 7)
80     print(f"Area of square (7x7): {sq_area}")
81
82     circ_area = calculate_area("circle", 5)
83     print(f"Area of circle (radius 5): {circ_area:.2f}")
84

```

```

13.3(1).py > ...
84
85     # Example of error handling
86     try:
87         calculate_area("triangle", 10, 5)
88     except ValueError as e:
89         print(f"\nError caught as expected: {e}")
90
91     try:
92         calculate_area("square", 10, 5)
93     except TypeError as e:
94         print(f"Error caught as expected: {e}")
95

```

OUTPUT:

```

PS C:\Users\DELL\Desktop\vs code\.vscode> & C:/Users/DELL/AppData/Local/anaconda3/python.exe "c:/Users/DELL/Desktop/vs code/.vscode/13.3(1).py"
Area of rectangle (10x4): 40
Area of square (7x7): 49
Area of circle (radius 5): 78.54

Error caught as expected: Unsupported shape: 'triangle'. Supported shapes are: ['rectangle', 'square', 'circle']
PS C:\Users\DELL\Desktop\vs code\.vscode>

```

OBSERVATIONS:

Key Strengths

- Excellent Design Pattern:** Using the `_SHAPE_FUNCTIONS` dictionary as a dispatch table is the biggest strength. It decouples the main logic from the individual calculations, making it easy to add new shapes without modifying the `calculate_area` function (adhering to the Open/Closed Principle).
- Robust Error Handling:**
 - The `ValueError` for unsupported shapes provides a helpful list of valid options.
 - The `try...except TypeError` block is a clever way to provide a more user-friendly error message when the wrong number of dimensions is supplied for a shape.
 - It gracefully handles case-insensitivity by converting the input `shape` to lowercase.
- Clarity and Readability:**
 - The use of type hints (`Dict`, `Callable`, `Numeric`) makes the function signatures and data structures self-documenting.
 - Private helper functions (e.g., `_calculate_rectangle_area`) neatly encapsulate the logic for each shape.
- Comprehensive Documentation:** The docstrings are excellent. They clearly explain what the function does, its parameters, what it returns, and what it might raise. The inclusion of runnable `doctest` examples is a fantastic touch.
- Good Practices:** The code avoids "magic numbers" by using `math.pi` and includes a standard `if __name__ == "__main__":` block for demonstration and testing, allowing the file to be used as both a script and a module.

Minor Suggestions for Refinement

Task Description #2 – Error Handling in Legacy Code

Task: Legacy function without proper error handling

Python Code

```

def read_file(filename):
    f = open(filename, "r")
    data = f.read()

```

```

    f.close()
    return data

```

Expected Output:

AI refactors with `with open()` and `try-except`.

PROMPT:

S SANJANA SIRIVELLA

Legacy function without proper error handling Python Code def

read_file(filename): f = open(filename, "r")
data = f.read() f.close() return data.give me the code

CODE:

```
ai13.3(2).py > ...
1  import os
2  from typing import Optional
3
4  def read_file_safely(filepath: str) -> Optional[str]:
5      """
6      Reads data from a file safely and handles common I/O errors.
7
8      This function uses a 'with' statement to ensure the file is
9      always closed, even if an error occurs during reading.
10
11     Args:
12         filepath (str): The path to the file to be read.
13
14     Returns:
15         Optional[str]: The content of the file as a string if successful,
16         otherwise None if an error occurs.
17     """
18     try:
19         # The 'with' statement ensures the file is automatically closed.
20         # Specifying 'encoding' is a best practice to avoid issues on different systems.
21         with open(filepath, "r", encoding="utf-8") as f:
22             return f.read()
23     except FileNotFoundError:
24         print(f"Error: The file at '{filepath}' was not found.")
25         return None
26     except IOError as e:
27         # Catches other I/O errors like permission issues.
28         print(f"Error reading file at '{filepath}': {e}")
29         return None
30     except Exception as e:
```

```

4 def read_file_safely(filepath: str) -> Optional[str]:
30     except Exception as e:
31         # Catch any other unexpected errors.
32         print(f"An unexpected error occurred: {e}")
33         return None
34
35 # --- Example Usage ---
36 if __name__ == "__main__":
37     # Create a dummy file for the success case
38     dummy_filename = "test_file.txt"
39     with open(dummy_filename, "w", encoding="utf-8") as f:
40         f.write("Hello, this is the content of the test file.")
41
42     # 1. Test the success case
43     print("--- Testing successful read ---")
44     content = read_file_safely(dummy_filename)
45     if content:
46         print(f"Successfully read content:\n---\n{content}\n---")
47
48     # 2. Test the file not found case
49     print("\n--- Testing file not found ---")
50     read_file_safely("non_existent_file.txt")
51
52     # Clean up the dummy file
53     os.remove(dummy_filename)
54

```

OUTPUT:

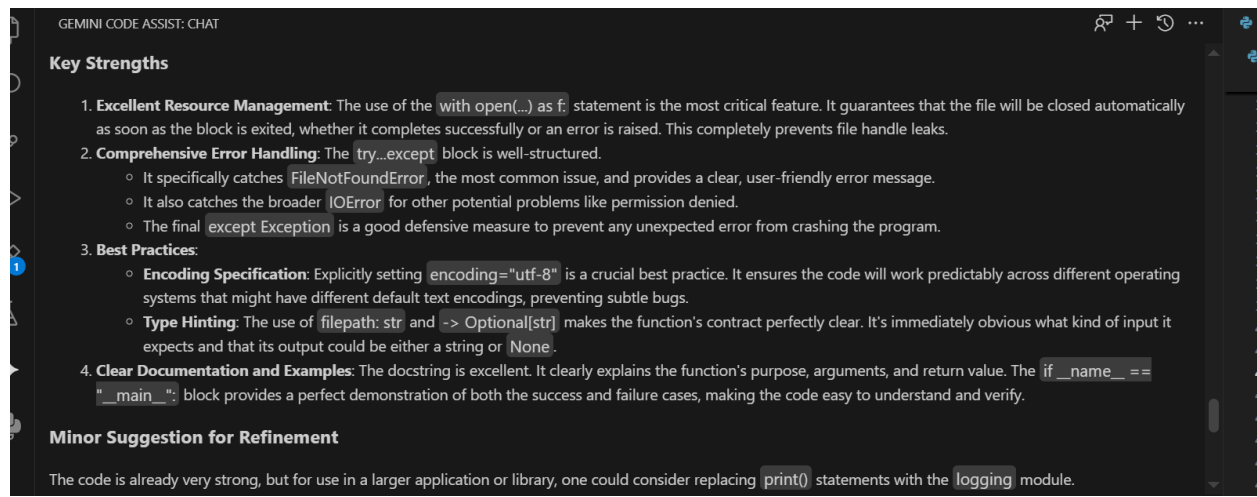
```

cle']
PS C:\Users\DELL\Desktop\vs code\.vscode> & C:/Users/DELL/AppData/Local/anaconda3/python.exe "c:/Users/DELL/Desktop/vs code/.vscode/ai13.3(2).py"
--- Testing successful read ---
Successfully read content:
---
Hello, this is the content of the test file.
Hello, this is the content of the test file.
---

--- Testing file not found ---
Error: The file at 'non_existent_file.txt' was not found.
PS C:\Users\DELL\Desktop\vs code\.vscode>

```

OBSERVATIONS:



Task Description #3 – Complex Refactoring

Task: Provide this legacy class to AI for readability and modularity improvements:

Python Code

class Student:

```
def __init__(self, n, a, m1, m2, m3):
```

```
    self.n = n
```

```
    self.a = a
```

```
    self.m1 = m1
```

```
    self.m2 = m2
```

```
    self.m3 = m3
```

```
    def details(self):
```

```
        print("Name:", self.n, "Age:", self.a)
```

```
    def total(self):
```

```
        return self.m1+self.m2+self.m3
```

Expected Output:

- AI improves naming (name, age, marks).
- Adds docstrings.
- Improves print readability.
- Possibly uses `sum(self.marks)` if marks stored in a list.

PROMPT:

S

SANJANA SIRIVELLA

Provide this legacy class to AI for readability and modularity improvements:
Python Code class Student: def **init**(self, n, a, m1, m2, m3): self.n = n self.a = a self.m1 = m1 self.m2 = m2 self.m3 = m3 def details(self): print("Name:", self.n, "Age:", self.a) def total(self): return self.m1+self.m2+self.m3.give me the code

CODE:

```
13-3-3.py > ...
1  from typing import List, Union
2
3  # Define a reusable type for numeric values
4  Numeric = Union[int, float]
5
6  class Student:
7      """
8      A well-structured class to represent a student and their marks.
9
10     This class encapsulates student data and provides methods to compute
11     derived information like total and average marks.
12     """
13
14     def __init__(self, name: str, age: int, marks: List[Numeric]):
15         """
16         Initializes a Student object.
17
18         Args:
19             name (str): The full name of the student.
20             age (int): The age of the student.
21             marks (List[Numeric]): A list of marks obtained by the student.
22
23         Raises:
24             ValueError: If the provided name is empty, age is not positive,
25                         or marks is not a list of numbers.
26         """
27         if not isinstance(name, str) or not name.strip():
28             raise ValueError("Name must be a non-empty string.")
29         if not isinstance(age, int) or age <= 0:
30             raise ValueError("Age must be a positive integer.")
```



```

6 class Student:
14     def __init__(self, name: str, age: int, marks: List[Numeric]):
20
27         if not isinstance(name, str) or not name.strip():
28             raise ValueError("Name must be a non-empty string.")
29         if not isinstance(age, int) or age <= 0:
30             raise ValueError("Age must be a positive integer.")
31         if not isinstance(marks, list):
32             raise ValueError("Marks must be provided as a list.")
33
34         self.name = name
35         self.age = age
36         self.marks = marks
37
38     @property
39     def total_marks(self) -> Numeric:
40         """Calculates and returns the sum of all marks."""
41         return sum(self.marks)
42
43     @property
44     def average_mark(self) -> float:
45         """
46         Calculates and returns the average of the marks.
47         Returns 0.0 if the marks list is empty to avoid division by zero.
48         """
49         if not self.marks:
50             return 0.0
51         return sum(self.marks) / len(self.marks)
52
53     def __str__(self) -> str:

```

```

6 class Student:
52
53     def __str__(self) -> str:
54         """Returns a user-friendly string representation of the student."""
55         return f"Student: {self.name}, Age: {self.age}"
56
57     def __repr__(self) -> str:
58         """Returns an unambiguous, developer-friendly representation."""
59         return f"Student(name='{self.name}', age={self.age}, marks={self.marks})"
60
61 # --- Example Usage ---
62 if __name__ == "__main__":
63     # Create an instance of the refactored Student class
64     student1 = Student(name="Alice", age=21, marks=[88, 92, 78, 95])
65
66     # --- Demonstrating the improved interface ---
67
68     # 1. The __str__ method is called by print() for a clean output
69     print(student1)
70
71     # 2. The __repr__ method is used for debugging or in the interactive console
72     print(repr(student1))
73
74     # 3. Accessing properties is clean and intuitive
75     print(f"Total Marks: {student1.total_marks}")
76     print(f"Average Mark: {student1.average_mark:.2f}")
77
78     # 4. The object's attributes are still directly accessible
79     print(f"First mark: {student1.marks[0]}")
80

```

```

80
81     # 5. Example of error handling for invalid data
82     try:
83         invalid_student = Student(name="", age=-5, marks=[])
84     except ValueError as e:
85         print(f"\nCaught expected error: {e}")
86

```

OUTPUT:

```

PS C:\Users\DELL\Desktop\vs code\.vscode> & C:/Users/DELL/AppData/Local/anaconda3/python.exe "c:/Users/DELL/Desktop/vs code/.vscode/13-3-3.py"
Student: Alice, Age: 21
Student(name='Alice', age=21, marks=[88, 92, 78, 95])
Total Marks: 353
Average Mark: 88.25
First mark: 88

Student: Alice, Age: 21
Student(name='Alice', age=21, marks=[88, 92, 78, 95])
Total Marks: 353
Average Mark: 88.25
First mark: 88

First mark: 88

Caught expected error: Name must be a non-empty string.
PS C:\Users\DELL\Desktop\vs code\.vscode>

```

OBSERVATIONS:

Key Strengths

- Excellent Readability and Naming:** The class and its attributes (`name`, `age`, `marks`) are descriptively named, making the code's intent immediately clear. This is a huge improvement over single-letter variable names.
- Flexible Data Structure:** Using a `List[Numeric]` for `marks` is a major strength. It makes the class flexible and scalable, allowing a student to have any number of marks, unlike a design with fixed attributes like `mark1`, `mark2`, etc.
- Use of Properties (@property):** The use of `@property` for `total_marks` and `average_mark` is a superb design choice. It allows derived values to be accessed like attributes (e.g., `student.average_mark`) rather than methods (`student.average_mark()`), which provides a cleaner and more intuitive API.
- Robustness and Error Handling:**
 - The `__init__` method performs validation on its inputs, raising a `ValueError` with a clear message if the data is invalid. This makes the class more reliable and prevents the creation of objects in an invalid state.
 - The `average_mark` property correctly handles the edge case of an empty `marks` list, preventing a `ZeroDivisionError`.
- Proper Object Representation (`__str__` and `__repr__`):** The implementation of both dunder methods is a hallmark of a well-designed Python class.
 - `__str__` provides a clean, user-friendly output suitable for `print()`.
 - `__repr__` provides an unambiguous, developer-focused representation that is invaluable for debugging.
- Excellent Documentation:** The class and its methods have clear docstrings and type hints, making the code self-documenting and easy for others to understand.

Prompts to try

- `__repr__` provides an unambiguous, developer-focused representation that is invaluable for debugging.
6. **Excellent Documentation:** The class and its methods have clear docstrings and type hints, making the code self-documenting and easy for others to understand and use correctly.
 7. **Clear Example Usage:** The `if __name__ == "__main__":` block provides a perfect demonstration of how to instantiate and interact with the class, including how to handle the errors it might raise.

Minor Suggestion for Refinement

The code is already excellent, but one very minor enhancement could be to validate the contents of the `marks` list during initialization. The current code checks if `marks` is a list, but not if its elements are numbers. The `sum()` function will eventually raise a `TypeError` if non-numeric data is present, but validating it in the constructor can provide a more immediate and specific error message.

Task Description #4 – Inefficient Loop Refactoring

Task: Refactor this inefficient loop with AI help

Python Code

```
nums = [1,2,3,4,5,6,7,8,9,10]
```

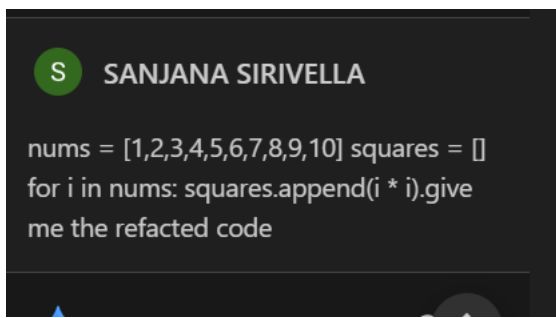
```
squares = []
```

```
for i in nums:
```

```
    squares.append(i * i)
```

Expected Output: AI suggested a list comprehension

PROMPT:



CODE:

```

13-3-4.py > ...
1  # List of numbers from 1 to 10
2  nums = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
3
4  squares = [i * i for i in nums]
5
6  # Printing the original list of numbers
7  print("Original numbers:", nums)
8
9  # Printing the list of squares
10 print("Squares of the numbers:", squares)
11

```

OUTPUT:

```

> & C:/Users/DELL/AppData/Local/anaconda3/python.exe "c:/Users/DELL/Desktop/vs code/.vscode/13-3-4.py"
Original numbers: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
Squares of the numbers: [1, 4, 9, 16, 25, 36, 49, 64, 81, 100]
PS C:\Users\DELL\Desktop\vs code\.vscode>

```

OBSERVATIONS:

Key Strengths

1. **Pythonic and Concise:** The most notable feature is the use of a list comprehension: `squares = [i * i for i in nums]`. This is a hallmark of modern Python. It's more readable and compact than an equivalent `for` loop with `.append()`.
2. **Clarity:** The code is very easy to understand. The variable names (`nums`, `squares`) are descriptive, and the comments clearly state the purpose of the print statements.
3. **Efficiency:** For creating a new list based on an existing one, a list comprehension is generally more performant than a manual `for` loop because the iteration is handled at a lower level in the Python interpreter.

Minor Suggestion for Refinement

The code is already great. A small suggestion for generating sequential numbers is to use Python's built-in `range()` function. While it makes no practical difference for a list of 10 items, it's a more scalable and conventional way to create number sequences.